

Unit I

1. Introduction

Artificial Intelligence:

In today's rapidly advancing technological landscape, AI has become a household term. From chatbots and virtual assistants to self-driving cars and recommendation algorithms, the impact of AI is ubiquitous. But what exactly is AI and how does it work?

What is Artificial Intelligence?

- It is a branch of Computer Science that pursues creating the computers or machines as intelligent as human beings. It is the science and engineering of making intelligent machines, especially intelligent computer programs.
- It is related to the similar task of using computers to understand human intelligence, but **AI** does not have to confine itself to methods that are biologically observable

Definition: Artificial Intelligence is the study of how to make computers do things, which, at the moment, people do better.

- According to the father of Artificial Intelligence, John McCarthy, it is *"The science and engineering of making intelligent machines, especially intelligent computer programs"*.
- Artificial Intelligence is a way of **making a computer, a computer-controlled robot, or a software think intelligently**, in the similar manner the intelligent humans think.
- AI is accomplished by studying how human brain thinks and how humans learn, decide, and work while trying to solve a problem, and then using the outcomes of this study as a basis of developing intelligent software and systems.
- From a **business** perspective AI is a set of very powerful tools, and methodologies for using those tools to solve business problems.
- From a **programming** perspective, AI includes the study of symbolic programming, problem solving, and search.

At its core, Artificial Intelligence refers to the simulation of human intelligence in machines that are programmed to think, reason, and learn like humans. Rather than being explicitly programmed for specific tasks, AI(Artificial Intelligence) systems use algorithms and vast amounts of data to recognize patterns, make decisions, and improve their performance over time.

Artificial Intelligence encompasses a wide range of technologies, including machine learning, natural language processing, computer vision, and robotics. These technologies enable AI systems to perform complex tasks, such as speech recognition and face detection, with remarkable accuracy.

Core Concepts in AI

Artificial Intelligence (AI) operates on a core set of concepts and technologies that enable machines to perform tasks that typically require human intelligence.

Here are some foundational concepts:

Machine Learning (ML): This is the backbone of AI, where algorithms learn from data without being explicitly programmed. It involves training an algorithm on a data set, allowing it to improve over time and make predictions or decisions based on new data.

Neural Networks: Inspired by the human brain, these are networks of algorithms that mimic the way neurons interact, allowing computers to recognize patterns and solve common problems in the fields of AI, machine learning, and deep learning.

Deep Learning: A subset of ML, deep learning uses complex neural networks with many layers (hence “deep”) to analyze various factors of data. This is instrumental in tasks like image and speech recognition.

Natural Language Processing (NLP): NLP involves programming computers to process and analyze large amounts of natural language data, enabling interactions between computers and humans using natural language.

Robotics: While often associated with AI, robotics merges AI concepts with physical components to create machines capable of performing a variety of tasks, from assembly lines to complex surgeries.

Cognitive Computing: This AI approach mimics human brain processes to solve complex problems, often using pattern recognition, NLP, and data mining.

Expert Systems: These are AI systems that emulate the decision-making ability of a human expert, applying reasoning capabilities to reach conclusions.

Each of these concepts helps to build systems that can automate, enhance, and sometimes outperform human capabilities in specific tasks.

How Does AI Work?

Artificial intelligence (AI) enables machines to learn from data and recognize patterns in it, to perform tasks more efficiently and effectively. AI works in five steps:

Input: Data is collected from various sources. This data is then sorted into categories.

Processing: The AI sorts and deciphers the data using patterns it has been programmed to learn until it recognizes similar patterns in the data.

Outcomes: The AI can then use those patterns to predict outcomes.

Adjustments: If the data sets are considered a “fail,” AI learns from that mistake, and the process is repeated again under different conditions.

Assessments: In this way, AI is constantly learning and improving.

Here are some of the key benefits of AI:

Improved Efficiency and Productivity: AI-powered systems can perform tasks with greater speed, accuracy, and consistency than humans, leading to improved efficiency and productivity in various industries. This can result in cost savings, reduced errors, and increased output.

Enhanced Decision-Making: AI algorithms can analyze large amounts of data, identify patterns, and make informed decisions faster than humans. This can be particularly useful in fields such as finance, healthcare, and logistics, where timely and accurate decision-making is critical.

Personalization and Customization: AI-powered systems can learn from user behavior and preferences to provide personalized recommendations, content, and experiences. This can lead to increased customer satisfaction and loyalty, as well as improved targeting and marketing strategies.

Automation of Repetitive Tasks: AI can be used to automate repetitive, time-consuming tasks, freeing up human resources to focus on more strategic and creative work. This can lead to cost savings, reduced errors, and improved work-life balance for employees.

Improved Safety and Risk Mitigation: AI-powered systems can be used to enhance safety in various applications, such as autonomous vehicles, industrial automation, and medical diagnostics. AI algorithms can also be used to detect and mitigate risks, such as fraud, cybersecurity threats, and environmental hazards.

Advancements in Scientific Research: AI can assist in scientific research by analyzing large datasets, generating hypotheses, and accelerating the discovery of new insights and breakthroughs. This can lead to advancements in fields such as medicine, climate science, and materials science.

Enhanced Human Capabilities: AI can be used to augment and enhance human capabilities, such as improving memory, cognitive abilities, and decision-making. This can lead to improved productivity, creativity, and problem-solving skills.

While the benefits of AI are numerous, it is important to consider the potential challenges and limitations of the technology, as well as the ethical implications of its use.

Challenges in Artificial Intelligence

While Artificial Intelligence (AI) has brought about numerous benefits and advancements, it also faces several challenges and limitations that must be addressed.

Here are some of the key challenges and limitations of AI:

Data Availability and Quality: AI systems rely on vast amounts of high-quality data to learn and make accurate predictions. However, obtaining and curating such data can be a significant challenge, particularly in domains where data is scarce or difficult to collect.

Bias and Fairness: AI algorithms can perpetuate and amplify biases present in the data used to train them, leading to decisions and outputs that are unfair or discriminatory. Addressing algorithmic bias is a crucial challenge in the development and deployment of AI systems.

Interpretability and Explainability: Many modern AI systems, such as deep learning models, are complex and opaque, making it difficult to understand how they arrive at their decisions. This lack of interpretability can be a significant barrier to trust and adoption, particularly in sensitive domains like healthcare and finance.

Safety and Robustness: AI systems can be vulnerable to adversarial attacks, where small, imperceptible changes to the input can cause the system to make erroneous or even dangerous decisions. Ensuring the safety and robustness of AI systems is a critical challenge.

Privacy and Security: The collection and use of personal data by AI systems raises significant privacy concerns, especially as the technology becomes more pervasive. Balancing the benefits of AI with the need to protect individual privacy is an ongoing challenge.

Scalability and Computational Limitations: Some AI algorithms and models can be computationally intensive, requiring significant computing power and resources. Scaling these systems to larger-scale applications can be a challenge, particularly in resource-constrained environments.

Ethical Considerations: The development and deployment of AI systems raise complex ethical questions, such as the impact on employment, the accountability for AI-driven decisions, and the potential for AI to be used for malicious purposes. Addressing these ethical concerns is crucial for the responsible and trustworthy use of AI.

As the field of AI continues to evolve, researchers and practitioners must work to address these challenges and limitations, ensuring that the technology is developed and deployed in a responsible and ethical manner.

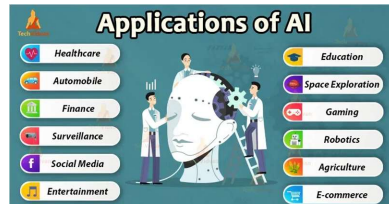
1.3 Applications of AI:

Artificial Intelligence has many practical applications across various industries and domains, including:

Healthcare – AI is used for medical diagnosis by analyzing medical images like X-rays and MRIs to identify diseases. For instance, AI systems are being developed to detect skin cancer from images with high accuracy.

Finance – AI helps in credit scoring by analyzing a borrower's financial history and other data to predict their creditworthiness. This helps banks decide whether to approve a loan and at what interest rate.

Retail – AI is used for product recommendations by analyzing your past purchases and browsing behavior to suggest products you might be interested in. For example, Amazon uses AI to recommend products to customers on their website.



Manufacturing – AI helps in quality control by inspecting products for defects. AI systems can be trained to identify even very small defects that human inspectors might miss.

Transportation – AI is used for autonomous vehicles by developing self-driving cars that can navigate roads without human input. Companies like Waymo and Tesla are developing self-driving car technology.

Customer service – AI-powered chatbots are used to answer customer questions and provide support. For instance, many banks use chatbots to answer customer questions about their accounts and transactions.

Security – AI is used for facial recognition by identifying people from images or videos. This technology is used for security purposes, such as identifying criminals or unauthorized individuals.

Marketing – AI is used for targeted advertising by showing ads to people who are most likely to be interested in the product or service being advertised. For example, social media companies use AI to target ads to users based on their interests and demographics.

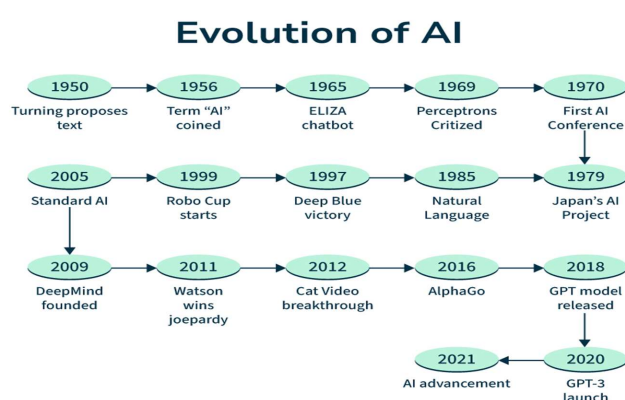
Education – AI is used for personalized learning by tailoring educational content to the individual needs of each student. For example, AI-powered tutoring systems can provide students with personalized instruction and feedback.

Need for Artificial Intelligence – Why is AI Important?

The widespread adoption of Artificial Intelligence (AI) has brought about numerous benefits and advantages across various industries and aspects of our lives.

1.2 History and Evolution of AI

This article examines the intriguing development of Artificial Intelligence from, its inception to its present state of development and promising prospects.



Fundamentally, [Artificial Intelligence](#) is the process of building machines that can replicate human intelligence. These machines can learn, reason, and adapt while carrying out activities that normally call for human intelligence. With artificial intelligence (AI) this world of natural language comprehension, image recognition, and decision making by computers can become a reality.

The Dawn of Artificial Intelligence (1950s-1960s)

The 1950s, which saw the following advancements, are considered to be the birthplace of AI :

- **1950:** In 1950 saw the publication of Alan Turing's work, "**Computing Machinery and Intelligence** " which introduced the Turing Test—a measure of computer intelligence.
- **1956:** A significant turning point in AI research occurs in 1956 when, **John McCarthy** first uses the phrase "*Artificial Intelligence*" at the Dartmouth Workshop.
- **1950s–1960s:** The goal of early artificial intelligence (AI) research was to encode human knowledge into computer programs through the use of symbolic reasoning, and logic-based environments.
- **Limited Advancement:** Quick advances are hampered by limited resources and computing-capacity.
- **Early AI systems:** This made an effort to encode human knowledge through the use of logic, and symbolic thinking. The development of early artificial intelligence (AI) systems that, depended on symbolic thinking and logic was hampered by a lack of resources, and processing capacity, which caused the field to advance slowly in the beginning.

AI's Early Achievements and Setbacks (1970s-1980s)

This age has seen notable developments as well as difficulties:

- **1970:** The 1970s witnessed the development of expert systems , which were intended to capture the knowledge of experts in a variety of domains. [Data Scientists](#) created rule-based systems that , could use pre-established guidelines to address certain issues.
- **Limitations:** Due to their inability to handle ambiguity and complicated circumstances , these systems had a limited range of applications.
- **The Artificial Intelligence Winter (1970–1980):** A period of inactivity brought on by a lack of funding, and un-met expectations.

Machine Learning and Data-Driven Approaches (1990s)

The 1990s bring a transformative move in AI:

- **1990s:** A worldview move towards [machine learning](#) approaches happens.
- **Rise of Machine-Learning:** Calculations learn from information utilizing strategies like neural systems, choice trees , and bolster vector machines.
- **Neural Organize Insurgency:** Propelled by the human brain, neural systems pick up ubiquity for errands like discourse acknowledgment, stock advertise expectation , and motion picture suggestions.
- **Information Powers AI:** Expanded handling control, and information accessibility fuel the development of data driven AI.
- **Unused Areas Rise:** Proposal frameworks, picture acknowledgment and normal dialect handling (NLP) take root.
- **Brilliant Age of AI:** [AI frameworks](#) exceed expectations in dis-course acknowledgment, stock determining, and suggestion frameworks.
- **Improved Execution:** Handling control enhancements and information accessibility drive progressions.

The AI Boom: Deep Learning and Neural Networks (2000s-2010s)

The 21st century, witnesses the rise of profound learning, and neural systems:

- **2000s-2010s:** Profound learning a subset of machine learning imitating the human brain's structure and work, came to the cutting edge.

- **Profound Neural Systems:** Multi-layered neural systems exceeded expectations in ranges such as - picture acknowledgment, NLP and gaming.
- **Innovative Progressions:** Profound learning encouraged advance in discourse acknowledgment, NLP, and computer vision.
- **Corporate Speculation:** Tech monsters like *Facebook, Google, and OpenAI* made noteworthy commitments to AI inquire about.
- **Counterfeit Neural Systems:** Complex calculations, based on interconnected neurons control profound learning headways.

Generative Pre-Trained Transformers: A New Era (GPT Series)

A novel advancement in recent times is the use of Generative Pre-Trained Transformers:

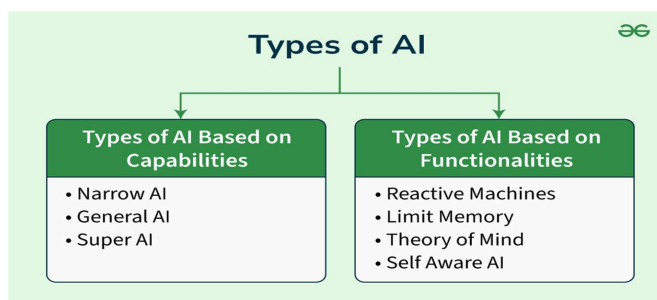
- **GPT Series:** Trained on enormous volumes of textual data, these models have rocked the globe.
- **GPT-3:** This model transforms language processing by producing writing that is similar to that of a human being and translating between languages.
- **Learning from Text:** Large volumes of text are absorbed by GPT models, such as - [GPT-3](#), which help them comprehend syntax, context, and comedy.
- **Beyond Translation:** GPT-3 serves as a portable writing assistant by producing essays, poetry, and even language translations.
- **The Upcoming Generation:** This new wave of models, which can write, translate and generate original material as well as provide insightful responses, is exemplified by models such as Bard, [ChatGPT](#), and Bing Co-pilot.
- **Pushing Boundaries:** These developments have increased the possible applications of AI showcasing its ability in content production, creative projects and language translation.

Types of Artificial Intelligence

Artificial Intelligence (AI) has transformed industries, leading to significant advancements in technology, science, and everyday life. To understand AI better, we must first recognize that AI can be categorized into different types based on capabilities and functionalities.

Type 1: [Based on Capabilities of AI](#) Type 2: [Based on the Functionality of AI](#)

- | | |
|--------------|-------------------|
| • Narrow AI | Reactive Machines |
| • General AI | Limited Memory AI |
| • Super AI | Theory of Mind |
| | Self-Aware AI |



Types of AI Based on Capabilities

1. Narrow AI (Weak AI)

[Narrow AI](#) is designed and trained on a specific task or a narrow range tasks. These Narrow AI systems are designed and trained for a purpose. These Narrow systems performs their designated tasks but mainly lack in the ability to generalize tasks.

Key Characteristics of Narrow AI

- **Task-Specific:** Narrow AI is built to perform particular tasks such as facial recognition, language translation, or playing chess.
- **No Generalization:** These systems cannot generalize their knowledge or apply it to tasks outside their designated function.
- **Human-Like Performance:** In their specialized domains, Narrow AI can perform at or even surpass human levels, but they do not possess understanding or consciousness.

Examples:

- Voice assistants like **Siri** or **Alexa** that understand specific commands.
- **Facial recognition** software used in security systems.
- **Recommendation engines** used by platforms like Netflix or Amazon.

Despite being highly efficient at specific tasks, Narrow AI lacks the ability to function beyond its predefined scope. These systems do not possess understanding or awareness.

2. General AI (Strong AI)

General AI refers to AI systems that have human intelligence and abilities to perform various tasks. Systems have capability to understand, learn and apply across a wide range of tasks that are similar to how a human can adapt to various tasks.

While General AI remains a theoretical concept, researchers aim to develop AI systems that can perform any intellectual task a human can. It requires the machine to have consciousness, self-awareness, and the ability to make independent decisions, which is not yet achievable.

Potential Applications:

- Robots that can learn new skills and adapt to unforeseen challenges in real-time.
- AI systems that could autonomously diagnose and solve complex medical issues across various specializations.

Key Characteristics of General AI

- **Broad Intelligence:** General AI would be able to perform a variety of tasks, not just one, making it versatile and adaptable.
- **Human-Like Reasoning:** It would have the ability to reason, solve problems, and make decisions just like a human being.
- **Self-Learning:** General AI would be capable of learning and improving over time, adapting to new situations and acquiring new skills without human intervention.

As of now, General AI remains theoretical and has not yet been achieved. Researchers are working on creating AI systems that could one day reach this level of capability, but it is considered a long-term goal in AI development.

3. Superintelligence (Super AI)

Super AI surpasses intelligence of human in solving-problem, creativity, and overall abilities. Super AI develops emotions, desires, need and beliefs of their own. They are able to make decisions of their own and solve problem of its own. Such AI would not only be able to complete tasks better than humans but also understand and interpret emotions and respond in a human-like manner.

While **Super AI** remains speculative, it could revolutionize industries, scientific research, and problem-solving, possibly leading to unprecedented advancements. However, it also raises ethical concerns regarding control and regulation.

Key Characteristics of Superintelligent AI

- **Surpasses Human Intelligence:** Superintelligent AI would exceed human cognitive abilities, potentially making it the most powerful tool or threat in existence.

- **Autonomous Decision-Making:** This AI would be able to make decisions without human input, and its reasoning and actions could be beyond human comprehension.
- **Ethical and Existential Concerns:** The development of Superintelligent AI raises significant ethical questions, including the potential risks it could pose to humanity if not properly controlled.

Like General AI, Superintelligent AI is still a concept explored in theory and science fiction. Its potential development is a subject of intense debate among AI researchers, ethicists, and futurists.

Types of Artificial Intelligence Based on Functionalities

AI can also be classified into four types based on how the systems function. This classification is more commonly used to distinguish AI systems in practical applications.

Here are the main types:

1. Reactive AI
2. Limited Memory AI
3. Theory of Mind AI
4. Self-Aware AI

Reactive AI: The Foundation of Artificial Intelligence

1. [Reactive AI](#) is the most basic type of AI. It is designed to respond to specific inputs with predetermined outputs and does not have the ability to form memories or learn from past experiences. Reactive AI operates solely on the present data it receives, making decisions based on immediate information.

Key Characteristics of Reactive AI

- **No Memory:** Reactive AI does not store any past data or experiences, so each interaction is treated as a new one.
- **Task-Specific:** It is designed to perform specific tasks and cannot adapt to new situations beyond its programming.
- **Lacks Understanding of Context:** This type of AI does not understand the broader context or the environment in which it operates.

Examples of Reactive AI systems

IBM's Deep Blue

The chess-playing computer that famously defeated world champion Garry Kasparov in 1997 is a classic example of Reactive AI. Deep Blue could evaluate a vast number of possible moves and counter-moves in the game but had no understanding of the game itself beyond the rules and its programming. It could not learn or improve from its experiences.

Google's AlphaGo

AlphaGo, developed by DeepMind, is a reactive AI that famously defeated the world champion Go player, Lee Sedol, in 2016. Like Deep Blue, AlphaGo could evaluate numerous possible moves and counter-moves in the game of Go. However, it lacked any understanding of the game beyond its programming and could not learn or improve from its past experiences during a game.

2. Limited Memory AI: Learning from the Past

Limited Memory AI builds upon Reactive AI by incorporating the ability to learn from historical data to make better decisions in the future. This type of AI can store past experiences and use them to influence future actions, making it more advanced and adaptable than Reactive AI.

Key Characteristics of Limited Memory AI

- **Memory-Dependent:** Limited Memory AI systems can retain and use past data to improve their decision-making processes.
- **Training Required:** These systems require training on large datasets to function effectively, as they learn patterns from historical data.

- **Improved Adaptability:** Unlike Reactive AI, Limited Memory AI can adapt to new information and scenarios, making it more versatile in dynamic environments.

Example of Limited Memory AI

Self-Driving Cars

Autonomous vehicles are a prominent example of Limited Memory AI. These cars are equipped with sensors and cameras that continuously gather data about the environment. They use this data, along with stored information from previous drives, to make real-time decisions such as when to stop, accelerate, or change lanes. The more data the car collects, the better it becomes at predicting and responding to various driving scenarios.

3. Theory of Mind AI: Understanding Human Emotions and Beliefs

Theory of Mind AI represents a more advanced type of AI that has the capability to understand and interpret human emotions, beliefs, intentions, and social interactions. This type of AI is still in the research and development phase, but it aims to create machines that can engage in more natural and meaningful interactions with humans.

Key Characteristics of Theory of Mind AI

- **Social Intelligence:** Theory of Mind AI is designed to understand and respond to human emotions and social cues, making interactions more personalized and effective.
- **Human-Like Understanding:** It can anticipate how humans might react in certain situations, leading to more intuitive and responsive AI systems.
- **Complex Decision-Making:** This type of AI can consider multiple variables, including emotional states and social contexts, when making decisions.

Example of Theory of Mind AI

Sophia the Robot

Developed by Hanson Robotics, Sophia is designed to engage in human-like conversations and simulate emotions through facial expressions and body language. Although her responses are scripted and based on pre-defined algorithms, Sophia represents an attempt to create robots that can interact socially and recognize human emotions.

Kismet

Developed at MIT Media Lab, Kismet is an early robot designed to interact with humans in a socially intelligent manner. It can recognize and respond to emotional cues through facial expressions and vocal tones, simulating the ability to understand and respond to human emotions.

4. Self-Aware AI: The Future of Artificial Intelligence

Self-aware AI represents the most advanced and theoretical type of AI. As the name suggests, self-aware AI systems would possess a level of consciousness similar to that of humans. They would be aware of their own existence, have the ability to form their own beliefs, desires, and emotions, and could potentially surpass human intelligence.

Key Characteristics of Self-Aware AI

- **Self-Consciousness:** These AI systems would have a sense of self, allowing them to understand their own existence and their place in the world.
- **Autonomous Decision-Making:** Self-aware AI would be capable of making decisions based on a deep understanding of itself and its environment.
- **Ethical Considerations:** The development of self-aware AI raises significant ethical questions, including the rights of such entities and the potential risks of creating machines that could surpass human intelligence.

Example of Self-Aware AI

Hypothetical Advanced AI Systems: Future AI systems that could possess self-awareness might be capable of introspection, understanding their own state, and making independent decisions based on self-interest. Such systems are often depicted in science fiction, like HAL 9000 from "2001: A Space Odyssey."

Theoretical AI in Research

Researchers in AI ethics and philosophy discuss the potential and implications of self-aware AI. These discussions involve theoretical frameworks for creating AI that understands its existence and possesses consciousness.

AI in Science Fiction

Characters like Skynet from the "Terminator" series, the AI in "Ex Machina," and other science fiction portrayals often depict self-aware AI. These fictional examples explore the ethical, philosophical, and practical challenges of creating machines with self-awareness.

1.4 AI Agents:

In artificial intelligence, an agent is a computer program or system that is designed to perceive its environment, make decisions and take actions to achieve a specific goal or set of goals. The agent operates autonomously, meaning it is not directly controlled by a human operator.

Agents can be classified into different types based on their characteristics, such as whether they are reactive or proactive, whether they have a fixed or dynamic environment, and whether they are single or multi-agent systems.

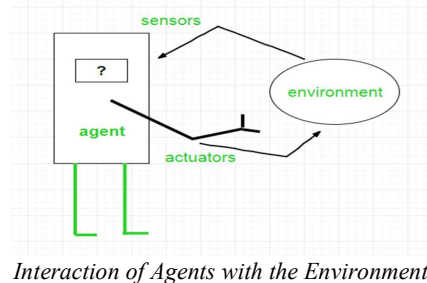
- Reactive agents are those that respond to immediate stimuli from their environment and take actions based on those stimuli. Proactive agents, on the other hand, take initiative and plan ahead to achieve their goals. The environment in which an agent operates can also be fixed or dynamic. Fixed environments have a static set of rules that do not change, while dynamic environments are constantly changing and require agents to adapt to new situations.
- Multi-agent systems involve multiple agents working together to achieve a common goal. These agents may have to coordinate their actions and communicate with each other to achieve their objectives. Agents are used in a variety of applications, including robotics, gaming, and intelligent systems. They can be implemented using different programming languages and techniques, including machine learning and natural language processing.

Artificial intelligence is defined as the study of rational agents. A rational agent could be anything that makes decisions, such as a person, firm, machine, or software. It carries out an action with the best outcome after considering past and current percepts (agent's perceptual inputs at a given instance). An AI system is composed of an **agent and its environment**. The agents act in their environment. The environment may contain other agents.

An agent is anything that can be viewed as:

- Perceiving its environment through **sensors** and
- Acting upon that environment through **actuators**

Note: Every agent can perceive its own actions (but not always the effects).



Interaction of Agents with the Environment

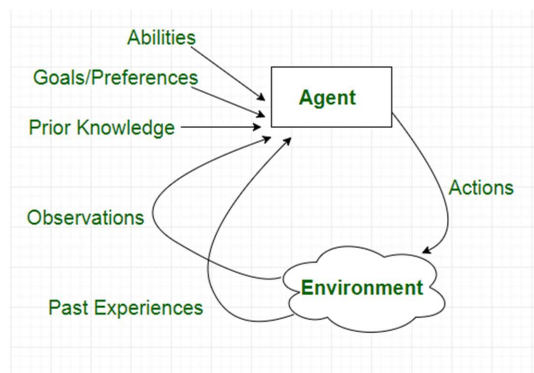
Structure of an AI Agent

To understand the structure of Intelligent Agents, we should be familiar with *Architecture* and *Agent* programs. **Architecture** is the machinery that the agent executes on. It is a device with sensors and actuators, for example, a robotic car, a camera, and a PC. **An agent program** is an implementation of an agent function. An **agent function** is a map from the percept sequence (history of all that an agent has perceived to date) to an action.

Agent = Architecture + Agent Program

There are many examples of agents in artificial intelligence. Here are a few:

- **Intelligent personal assistants:** These are agents that are designed to help users with various tasks, such as scheduling appointments, sending messages, and setting reminders. Examples of intelligent personal assistants include Siri, Alexa, and Google Assistant.
- **Autonomous robots:** These are agents that are designed to operate autonomously in the physical world. They can perform tasks such as cleaning, sorting, and delivering goods. Examples of autonomous robots include the Roomba vacuum cleaner and the Amazon delivery robot.
- **Gaming agents:** These are agents that are designed to play games, either against human opponents or other agents. Examples of gaming agents include chess-playing agents and poker-playing agents.
- **Fraud detection agents:** These are agents that are designed to detect fraudulent behavior in financial transactions. They can analyze patterns of behavior to identify suspicious activity and alert authorities. Examples of fraud detection agents include those used by banks and credit card companies.
- **Traffic management agents:** These are agents that are designed to manage traffic flow in cities. They can monitor traffic patterns, adjust traffic lights, and reroute vehicles to minimize congestion. Examples of traffic management agents include those used in smart cities around the world.
- A **software agent** has Keystrokes, file contents, received network packages that act as sensors and displays on the screen, files, and sent network packets acting as actuators.
- A **Human-agent** has eyes, ears, and other organs which act as sensors, and hands, legs, mouth, and other body parts act as actuators.
- A **Robotic agent** has Cameras and infrared range finders which act as sensors and various motors act as actuators.



Characteristics of an Agent

1.5 Types of Agents

Agents can be grouped into five classes based on their degree of perceived intelligence and capability:

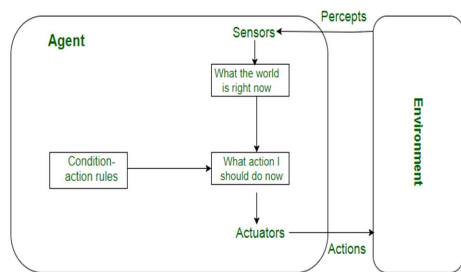
- Simple Reflex Agents
- Model-Based Reflex Agents
- Goal-Based Agents
- Utility-Based Agents
- Learning Agent
- Multi-agent systems
- Hierarchical agents

Simple Reflex Agents

Simple reflex agents ignore the rest of the percept history and act only on the basis of the **current percept**. Percept history is the history of all that an agent has perceived to date. The agent function is based on the **condition-action rule**. A condition-action rule is a rule that maps a state i.e., a condition to an action. If the condition is true, then the action is taken, else not. This agent function only succeeds when the environment is fully observable. For simple reflex agents operating in partially observable environments, infinite loops are often unavoidable. It may be possible to escape from infinite loops if the agent can randomize its actions.

Problems with Simple reflex agents are :

- Very limited intelligence.
- No knowledge of non-perceptual parts of the state.
- Usually too big to generate and store.
- If there occurs any change in the environment, then the collection of rules needs to be updated.



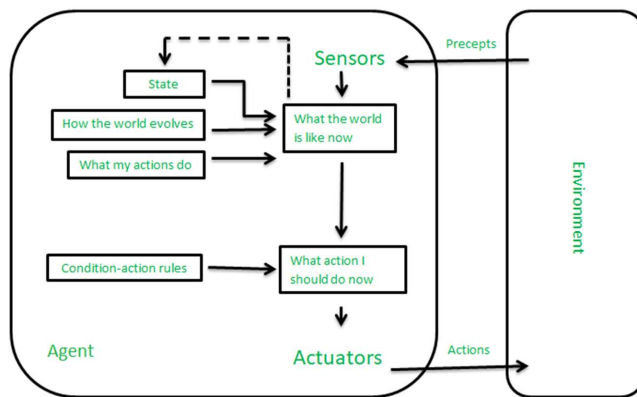
Simple Reflex Agents

Model-Based Reflex Agents

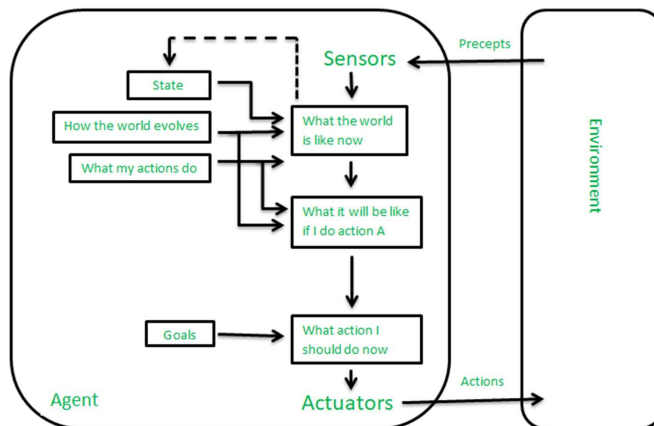
It works by finding a rule whose condition matches the current situation. A model-based agent can handle **partially observable environments** by the use of a model about the world. The agent has to keep track of the **internal state** which is adjusted by each percept and that depends on the percept history. The current state is stored inside the agent which maintains some kind of structure describing the part of the world which cannot be seen.

Updating the state requires information about:

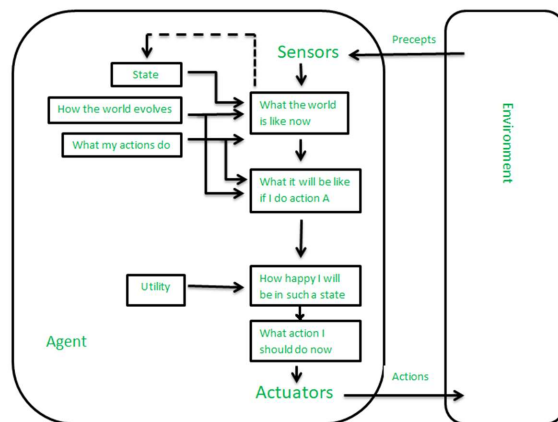
- How the world evolves independently from the agent?
- How do the agent's actions affect the world?

*Model-Based Reflex Agents***Goal-Based Agents**

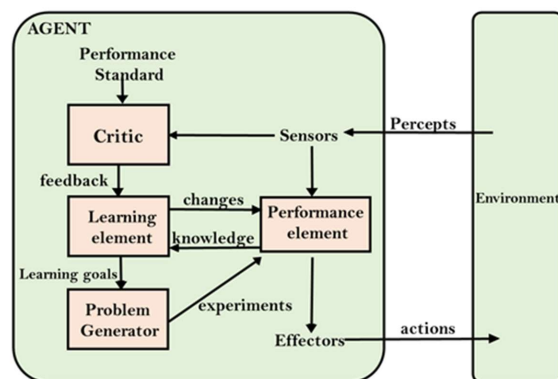
These kinds of agents take decisions based on how far they are currently from their **goal** (description of desirable situations). Their every action is intended to reduce their distance from the goal. This allows the agent a way to choose among multiple possibilities, selecting the one which reaches a goal state. The knowledge that supports its decisions is represented explicitly and can be modified, which makes these agents more flexible. They usually require search and planning. The goal-based agent's behavior can easily be changed.

*Goal-Based Agents***Utility-Based Agents**

The agents which are developed having their end uses as building blocks are called utility-based agents. When there are multiple possible alternatives, then to decide which one is best, utility-based agents are used. They choose actions based on a **preference (utility)** for each state. Sometimes achieving the desired goal is not enough. We may look for a quicker, safer, cheaper trip to reach a destination. Agent happiness should be taken into consideration. Utility describes how **“happy”** the agent is. Because of the uncertainty in the world, a utility agent chooses the action that maximizes the expected utility. A utility function maps a state onto a real number which describes the associated degree of happiness.

*Utility-Based Agents***Learning Agent**

A learning agent in AI is the type of agent that can learn from its past experiences or it has learning capabilities. It starts to act with basic knowledge and then is able to act and adapt automatically through learning. A learning agent has mainly four conceptual components, which are:

*Learning Agent*

1. **Learning element:** It is responsible for making improvements by learning from the environment.
2. **Critic:** The learning element takes feedback from critics which describes how well the agent is doing with respect to a fixed performance standard.
3. **Performance element:** It is responsible for selecting external action.
4. **Problem Generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.

Multi-Agent Systems

These agents interact with other agents to achieve a common goal. They may have to coordinate their actions and communicate with each other to achieve their objective.

A multi-agent system (MAS) is a system composed of multiple interacting agents that are designed to work together to achieve a common goal. These agents may be autonomous or semi-autonomous and are capable of perceiving their environment, making decisions, and taking action to achieve the common objective.

MAS can be used in a variety of applications, including transportation systems, robotics, and social networks. They can help improve efficiency, reduce costs, and increase flexibility in complex systems. MAS can be classified into different types based on their characteristics, such as whether the agents have the same or different goals, whether the agents are cooperative or competitive, and whether the agents are homogeneous or heterogeneous.

- In a homogeneous MAS, all the agents have the same capabilities, goals, and behaviors.
- In contrast, in a heterogeneous MAS, the agents have different capabilities, goals, and behaviors.

This can make coordination more challenging but can also lead to more flexible and robust systems. Cooperative MAS involves agents working together to achieve a common goal, while competitive MAS involves agents working against each other to achieve their own goals. In some cases, MAS can also involve both cooperative and competitive behavior, where agents must balance their own interests with the interests of the group.

MAS can be implemented using different techniques, such as [game theory](#), [machine learning](#), and agent-based modeling. Game theory is used to analyze strategic interactions between agents and predict their behavior. Machine learning is used to train agents to improve their decision-making capabilities over time. Agent-based modeling is used to simulate complex systems and study the interactions between agents.

Overall, multi-agent systems are a powerful tool in artificial intelligence that can help solve complex problems and improve efficiency in a variety of applications.

Hierarchical Agents

These agents are organized into a hierarchy, with high-level agents overseeing the behavior of lower-level agents. The high-level agents provide goals and constraints, while the low-level agents carry out specific tasks. Hierarchical agents are useful in complex environments with many tasks and sub-tasks.

- Hierarchical agents are agents that are organized into a hierarchy, with high-level agents overseeing the behavior of lower-level agents. The high-level agents provide goals and constraints, while the low-level agents carry out specific tasks. This structure allows for more efficient and organized decision-making in complex environments.
- Hierarchical agents can be implemented in a variety of applications, including robotics, manufacturing, and transportation systems. They are particularly useful in environments where there are many tasks and sub-tasks that need to be coordinated and prioritized.

Uses of Agents

Agents are used in a wide range of applications in artificial intelligence, including:

- **Robotics:** Agents can be used to control robots and automate tasks in manufacturing, transportation, and other industries.
- **Smart homes and buildings:** Agents can be used to control heating, lighting, and other systems in smart homes and buildings, optimizing energy use and improving comfort.
- **Transportation systems:** Agents can be used to manage traffic flow, optimize routes for autonomous vehicles, and improve logistics and supply chain management.
- **Healthcare:** Agents can be used to monitor patients, provide personalized treatment plans, and optimize healthcare resource allocation.

- **Finance:** Agents can be used for automated trading, fraud detection, and risk management in the financial industry.
- **Games:** Agents can be used to create intelligent opponents in games and simulations, providing a more challenging and realistic experience for players.
- **Natural language processing:** Agents can be used for language translation, question answering, and chatbots that can communicate with users in [natural language](#).
- **Cybersecurity:** Agents can be used for intrusion detection, malware analysis, and network security.
- **Environmental monitoring:** Agents can be used to monitor and manage natural resources, track climate change, and improve environmental sustainability.
- **Social media:** Agents can be used to analyze social media data, identify trends and patterns, and provide personalized recommendations to users.

Overall, agents are a versatile and powerful tool in [artificial intelligence](#) that can help solve a wide range of problems in different fields.

1.6 Search Algorithms in AI

Artificial Intelligence is the study of building agents that act rationally. Most of the time, these agents perform some kind of search algorithm in the background in order to achieve their tasks.

Introduction to Problem Solving, General problem solving

Problem solving is a process of generating solutions from observed data.

- a problem is characterized by a set of goals,
- a set of objects, and
- a set of operations.

These could be ill-defined and may evolve during problem solving.

Searching Solutions:

To build a system to solve a problem:

1. Define the problem precisely
2. Analyze the problem
3. Isolate and represent the task knowledge that is necessary to solve the problem
4. Choose the best problem-solving techniques and apply it to the particular problem.

Defining the problem as State Space Search:

The state space representation forms the basis of most of the AI methods.

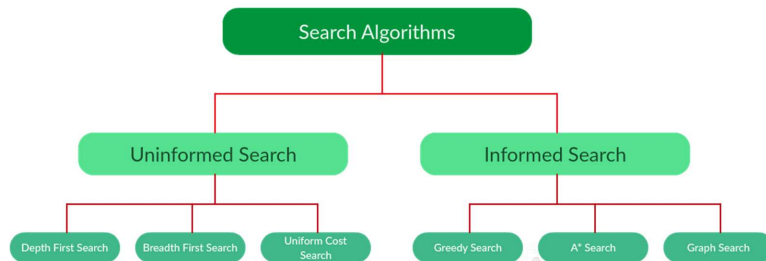
- Formulate a problem as a **state space search** by showing the legal problem states, the legal operators, and the initial and goal states.
- A **state** is defined by the specification of the values of all attributes of interest in the world
- An **operator** changes one state into the other; it has a precondition which is the value of certain attributes prior to the application of the operator, and a set of effects, which are the attributes altered by the operator
- The **initial state** is where you start
- The **goal state** is the partial description of the solution

A search problem consists of:

- **A State Space.** Set of all possible states where you can be.
- **A Start State.** The state from where the search begins.
- **A Goal State.** A function that looks at the current state returns whether or not it is the goal state.
- The **Solution** to a search problem is a sequence of actions, called the **plan** that transforms the start state to the goal state.
- This plan is achieved through search algorithms.

Types of search algorithms:

There are far too many powerful search algorithms out there to fit in a single article. Instead, this article will discuss *six* of the fundamental search algorithms, divided into *two* categories, as shown below.



Note that there is much more to search algorithms than the chart I have provided above. However, this article will mostly stick to the above chart, exploring the algorithms given there.

Uninformed Search Algorithms:

The search algorithms in this section have no additional information on the goal node other than the one provided in the problem definition. The plans to reach the goal state from the start state differ only by the order and/or length of actions. Uninformed search is also called **Blind search**. These algorithms can only generate the successors and differentiate between the goal state and non goal state.

The following uninformed search algorithms are discussed in this section.

1. Depth First Search
2. Breadth First Search
3. Uniform Cost Search

Each of these algorithms will have:

- A problem **graph**, containing the start node S and the goal node G.
- A **strategy**, describing the manner in which the graph will be traversed to get to G.
- A **fringe**, which is a data structure used to store all the possible states (nodes) that you can go from the current states.
- A **tree**, that results while traversing to the goal node.
- A solution **plan**, which the sequence of nodes from S to G.

The Water Jug Problem:

In this problem, we use two jugs called four and three; four holds a maximum of four gallons of water and three a maximum of three gallons of water. How can we get two gallons of water in the four jug? The state space is a set of prearranged pairs giving the number of gallons of water in the pair of jugs at any time, i.e., (four, three) where four = 0, 1, 2, 3 or 4 and three = 0, 1, 2 or 3.

The start state is (0, 0) and the goal state is (2, n) where n may be any but it is limited to three holding from 0 to 3 gallons of water or empty. Three and four shows the name and numerical number shows the amount of water in jugs for solving the water jug problem. The major production rules for solving this problem are shown below:

Initial condition	Goal comment
1. (four, three) if four < 4	(4, three) fill four from tap
2. (four, three) if three < 3	(four, 3) fill three from tap
3. (four, three) If four > 0	(0, three) empty four into drain
4. (four, three) if three > 0	(four, 0) empty three into drain

- | | |
|--------------------------------------|---|
| 5. (four, three) if four + three < 4 | (four + three, 0) empty three into four |
| 6. (four, three) if four + three < 3 | (0, four + three) empty four into three |
| 7. (0, three) If three > 0 | (three, 0) empty three into four |
| 8. (four, 0) if four > 0 | (0, four) empty four into three |
| 9. (0, 2) | (2, 0) empty three into four |
| 10. (2, 0) | (0, 2) empty four into three |
| 11. (four, three) if four < 4 | (4, three-diff) pour diff, 4-four, into four from three |
| 12. (three, four) if three < 3 | (four-diff, 3) pour diff, 3-three, into three from four and a solution is given below four three rule |

(Fig. 2.2 Production Rules for the Water Jug Problem)

Gallons in Four Jug	Gallons in Three Jug	Rules Applied
0	0	-
0	3	2
3	0	7
3	3	2
4	2	11
0	2	3
2	0	10

(Fig. 2.3 One Solution to the Water Jug Problem)

The problem solved by using the production rules in combination with an appropriate control strategy, moving through the problem space until a path from an initial state to a goal state is found. In this problem solving process, search is the fundamental concept. For simple problems it is easier to achieve this goal by hand but there will be cases where this is far too difficult.

PRODUCTION SYSTEMS

Production systems provide appropriate structures for performing and describing search processes. A production system has four basic components as enumerated below. A set of rules each consisting of a left side that determines the applicability of the rule and a right side that describes the operation to be performed if the rule is applied.

- A database of current facts established during the process of inference.
- A control strategy that specifies the order in which the rules will be compared with facts in the database and also specifies how to resolve conflicts in selection of several rules or selection of more facts.
- A rule firing module.

The production rules operate on the knowledge database. Each rule has a precondition that is, either satisfied or not by the knowledge database. If the precondition is satisfied, the rule can be applied. Application of the rule changes the knowledge database. The control system chooses which applicable rule should be applied and ceases computation when a termination condition on the knowledge database is satisfied.

Example: Eight puzzle (8-Puzzle)

The 8-puzzle is a 3×3 array containing eight square pieces, numbered 1 through 8, and one empty space. A piece can be moved horizontally or vertically into the empty space, in effect exchanging the positions of the piece and the empty space.

There are four possible moves, UP (move the blank space up), DOWN, LEFT and RIGHT. The aim of the game is to make a sequence of moves that will convert the board from the start state into the goal state:

2	3	4
8	6	2
7		5

Initial State

1	2	3
8		4
7	6	5

Goal State

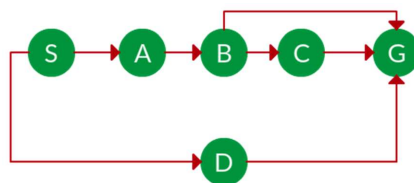
This example can be solved by the operator sequence UP, RIGHT, UP, LEFT, DOWN.

1. Depth First Search:

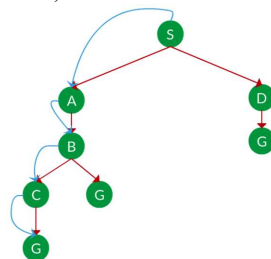
Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking. It uses last in- first-out strategy and hence it is implemented using a stack.

Example:

Question. Which solution would DFS find to move from node S to node G if run on the graph below?



Solution. The equivalent search tree for the above graph is as follows. As DFS traverses the tree “deepest node first”, it would always pick the deeper branch until it reaches the solution (or it runs out of nodes, and goes to the next branch). The traversal is shown in blue arrows.



Path: S -> A -> B -> C -> G

= the depth of the search tree = the number of levels of the search tree.

= number of nodes in level .

Time complexity: Equivalent to the number of nodes traversed in

DFS.

Space complexity: Equivalent to how large can the fringe get.

Completeness: DFS is complete if the search tree is finite, meaning for a given finite search tree, DFS will come up with a solution if it exists.

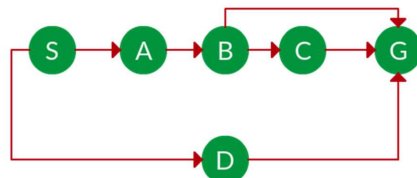
Optimality: DFS is not optimal, meaning the number of steps in reaching the solution, or the cost spent in reaching it is high.

2. Breadth First Search:

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level. It is implemented using a queue.

Example:

Question. Which solution would BFS find to move from node S to node G if run on the graph below?



Solution. The equivalent search tree for the above graph is as follows. As BFS traverses the tree “shallowest node first”, it would always pick the shallower branch until it reaches the solution (or it runs out of nodes, and goes to the next branch). The traversal is shown in blue arrows.

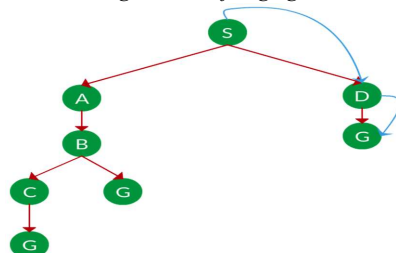
Path: S -> D -> G

= the depth of the shallowest solution.

= number of nodes in level .

Time complexity: Equivalent to the number of nodes traversed in BFS until the shallowest solution.

Space complexity: Equivalent to how large can the fringe get.



Completeness: BFS is complete, meaning for a given search tree, BFS will come up with a solution if it exists.

Optimality: BFS is optimal as long as the costs of all edges are equal.

3. Uniform Cost Search:

UCS is different from BFS and DFS because here the costs come into play. In other words, traversing via different edges might not have the same cost. The goal is to find a path where the cumulative sum of costs is the least.

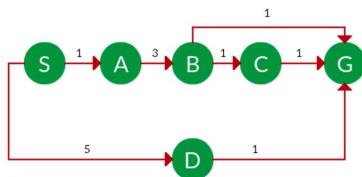
Cost of a node is defined as:

$\text{cost}(\text{node}) = \text{cumulative cost of all nodes from root}$

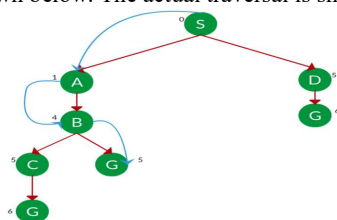
$\text{cost}(\text{root}) = 0$

Example:

Question. Which solution would UCS find to move from node S to node G if run on the graph below?



Solution. The equivalent search tree for the above graph is as follows. The cost of each node is the cumulative cost of reaching that node from the root. Based on the UCS strategy, the path with the least cumulative cost is chosen. Note that due to the many options in the fringe, the algorithm explores most of them so long as their cost is low, and discards them when a lower-cost path is found; these discarded traversals are not shown below. The actual traversal is shown in blue.



Path: $S \rightarrow A \rightarrow B \rightarrow G$

Cost: 5

Let C = cost of solution.

C = arcs cost.

Then C effective depth

Time complexity: , **Space complexity:**

Advantages:

- UCS is complete only if states are finite and there should be no loop with zero weight.
- UCS is optimal only if there is no negative cost.

Disadvantages:

- Explores options in every “direction”.
- No information on goal location.

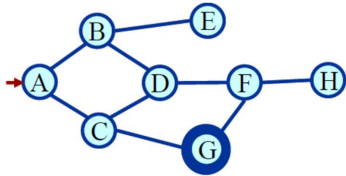
1. Breadth- First -Search:

Consider the state space of a problem that takes the form of a tree. Now, if we search the goal along each breadth of the tree, starting from the root and continuing up to the largest depth, we call it *breadth first search*.

• **Algorithm:**

1. Create a variable called NODE-LIST and set it to initial state
2. Until a goal state is found or NODE-LIST is empty do
 - a. Remove the first element from NODE-LIST and call it E. If NODE-LIST was empty, quit
 - b. For each way that each rule can match the state described in E do:
 - i. Apply the rule to generate a new state
 - ii. If the new state is a goal state, quit and return this state
 - iii. Otherwise, add the new state to the end of NODE-LIST

BFS illustrated:



Step 1: Initially fringe contains only one node corresponding to the source state A.

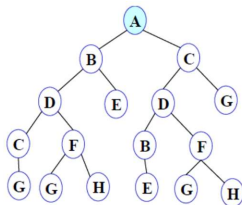


Fig:Step1

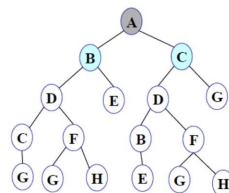


Fig: Step2

FRINGE: A

Step 2: A is removed from fringe. The node is expanded, and its children B and C are generated. They are placed at the back of fringe. **FRINGE: B C**

Step 3: Node B is removed from fringe and is expanded. Its children D, E are generated and put at the back of fringe. **FRINGE: C D E**

Commented [A1]:

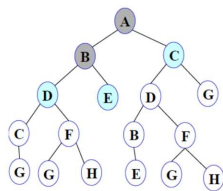


Fig:Step3

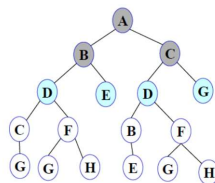


Fig: Step4

Step 4: Node C is removed from fringe and is expanded. Its children D and G are added to the back of fringe. **FRINGE: D E D G**

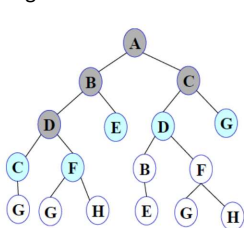


Fig:Step5

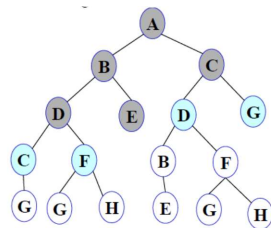


Fig: Step6

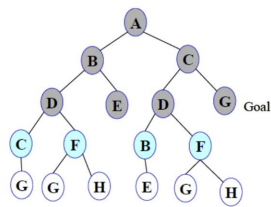


Fig: Step7

Step 5: Node D is removed from fringe. Its children C and F are generated and added to the back of fringe **FRINGE: E D G C F**

Step 6: Node E is removed from fringe. It has no children **FRINGE: D G C F**

Step 7: D is expanded; B and F are put in OPEN **FRINGE: G C F B F**

Step 8: G is selected for expansion. It is found to be a goal node. So the algorithm returns the path A C G by following the parent pointers of the node corresponding to G. The algorithm terminates.

Breadth first search is:

- One of the simplest search strategies
- Complete. If there is a solution, BFS is guaranteed to find it.
- If there are multiple solutions, then a minimal solution will be found
- The algorithm is optimal (i.e., admissible) if all operators have the same cost. Otherwise, breadth first search finds a solution with the shortest path length.
- **Time complexity:** $O(b^d)$
- **Space complexity:** $O(b^d)$
- **Optimality:** Yes b - branching factor (maximum no of successors of any node),

d – Depth of the shallowest goal node

Maximum length of any path (m) in search space

Advantages: Finds the path of minimal length to the goal

Disadvantages:

- Requires the generation and storage of a tree whose size is exponential the depth of the shallowest goal node.
- The breadth first search algorithm cannot be effectively used unless the search space is quite small.

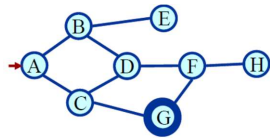
2. Depth- First- Search.

We may sometimes search the goal along the largest depth of the tree, and move up only when further traversal along the depth is not possible. We then attempt to find alternative offspring of the parent of the node (state) last visited. If we visit the nodes of a tree using the above principles to search the goal, the traversal made is called depth first traversal and consequently the search strategy is called *depth first search*.

• **Algorithm:**

1. Create a variable called NODE-LIST and set it to initial state
2. Until a goal state is found or NODE-LIST is empty do
 - a. Remove the first element from NODE-LIST and call it E. If NODE-LIST was empty, quit
 - b. For each way that each rule can match the state described in E do:
 - i. Apply the rule to generate a new state
 - ii. If the new state is a goal state, quit and return this state
 - iii. Otherwise, add the new state in front of NODE-LIST

DFS illustrated:



A State Space Graph

Step 1: Initially fringe contains only the node for A.

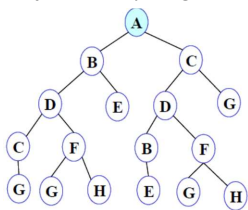


Fig: Step 1

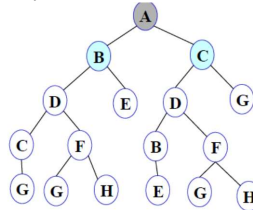


Fig: Step 2

FRINGE: A

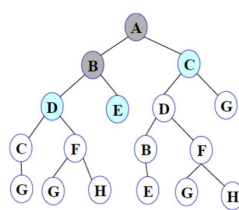


Fig: Step 3

Step 2: A is removed from fringe. A is expanded and its children B and C are put in front of fringe.

FRINGE: B C

Step 3: Node B is removed from fringe, and its children D and E are pushed in front of fringe.

FRINGE: D E C

Step 4: Node D is removed from fringe. C and F are pushed in front of fringe. **FRINGE: C F E C**

Step 5: Node C is removed from fringe. Its child G is pushed in front of fringe. **FRINGE: G F E C**

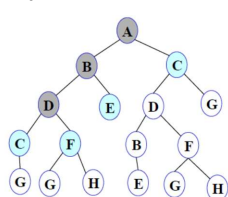


Fig: Step 4

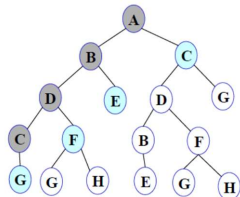


Fig: Step 5

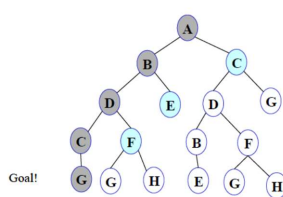


Fig: Step 6

Step 6: Node G is expanded and found to be a goal node. **FRINGE: G F E C**

The solution path A-B-D-C-G is returned and the algorithm terminates.

Depth first searches:

1. The algorithm takes exponential time.
2. If N is the maximum depth of a node in the search space, in the worst case the algorithm will take time $O(b^N)$.
3. The space taken is linear in the depth of the search tree, $O(bN)$.

Note that the time taken by the algorithm is related to the maximum depth of the search tree. If the search tree has infinite depth, the algorithm may not terminate. This can happen if the search space is infinite. It can also happen if the search space contains cycles. The latter case can be handled by checking for cycles in the algorithm. Thus **Depth First Search is not complete**.

3. Exhaustive searches- Iterative Deeping DFS

Description:

- It is a search strategy resulting when you combine BFS and DFS, thus combining the advantages of each strategy, taking the completeness and optimality of BFS and the modest memory requirements of DFS.
- IDS works by looking for the best search depth d , thus starting with depth limit 0 and make a BFS and if the search failed it increase the depth limit by 1 and try a BFS again with depth 1 and so on – first $d = 0$, then 1 then 2 and so on – until a depth d is reached where a goal is found.

Algorithm:

```

procedure IDDFS(root)
  for depth from 0 to  $\infty$ 
    found  $\leftarrow$  DLS(root, depth)
    if found  $\neq$  null
      return found
  procedure DLS(node, depth)
    if depth = 0 and node is a goal
      return node
    else if depth > 0
      foreach child of node
        found  $\leftarrow$  DLS(child, depth-1)
        if found  $\neq$  null
          return found
      return null
  
```

Performance Measure:

- Completeness: IDS is like BFS, is complete when the branching factor b is finite.
- Optimality: IDS is also like BFS optimal when the steps are of the same cost.
- Time Complexity:
- One may find that it is wasteful to generate nodes multiple times, but actually it is not that costly compared to BFS, that is because most of the generated nodes are always in the deepest level reached, consider that we are searching a binary tree and our depth limit reached 4, the nodes generated in last level = $2^4 = 16$, the nodes generated in all nodes before last level = $2^0 + 2^1 + 2^2 + 2^3 = 15$
- Imagine this scenario, we are performing IDS and the depth limit reached depth d , now if you remember the way IDS expands nodes, you can see that nodes at depth d are generated once, nodes at depth $d-1$ are generated 2 times, nodes at depth $d-2$ are generated 3 times and so on, until you reach depth 1 which is generated d times, we can view the total number of generated nodes in the worst case as:
- $\square N(\text{IDS}) = (b)d + (d-1)b^2 + (d-2)b^3 + \dots + (2)b^{d-1} + (1)b^d = O(b^d)$

If this search were to be done with BFS, the total number of generated nodes in the worst case will be like:

- $N(\text{BFS}) = b + b^2 + b^3 + b^4 + \dots + b^d + (bd + 1 - b) = O(bd + 1)$
- If we consider a realistic numbers, and use $b = 10$ and $d = 5$, then number of generated nodes in BFS and IDS will be like
 - $N(\text{IDS}) = 50 + 400 + 3000 + 20000 + 100000 = 123450$
 - $N(\text{BFS}) = 10 + 100 + 1000 + 10000 + 100000 + 999990 = 1111100$
 - BFS generates like 9 time nodes to those generated with IDS.

Space Complexity:

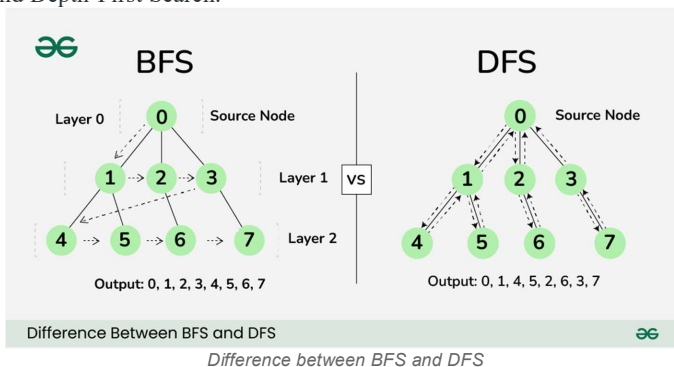
- IDS is like DFS in its space complexity, taking $O(bd)$ of memory.

Conclusion:

- We can conclude that IDS is a hybrid search strategy between BFS and DFS inheriting their advantages.
- IDS is faster than BFS and DFS.
- It is said that "IDS is the preferred uniformed search method when there is a large search space and the depth of the solution is not known".

1.7 Difference between BFS and DFS

[Breadth-First Search \(BFS\)](#) and [Depth-First Search \(DFS\)](#) are two fundamental algorithms used for traversing or searching graphs and trees. This article covers the basic difference between Breadth-First Search and Depth-First Search.



Parameters	BFS	DFS
Stands for	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
Data Structure	BFS(Breadth First Search) uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure.
Definition	BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level.	DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
Approach used	It works on the concept of FIFO (First In First Out).	It works on the concept of LIFO (Last In First Out).
Suitable for	BFS is more suitable for searching vertices closer to the given source.	DFS is more suitable when there are solutions away from source.
Applications	BFS is used in various applications such as bipartite graphs, shortest paths, etc. If weight of every edge is same, then BFS gives shortest path from source to every other vertex.	DFS is used in various applications such as acyclic graphs and finding strongly connected components etc. There are many applications where both BFS and DFS can be used like Topological Sorting, Cycle Detection, etc.

4. Depth Bonded Depth First Search(DBDFS):

Depth Limited Search is a key algorithm used in the problem space among the strategies concerned with artificial intelligence. The article provides a comprehensive overview of the Depth-Limited Search (DLS) algorithm, explaining its concept, applications, and implementation in solving pathfinding problems in robotics, while also addressing frequently asked questions.

Introducing Depth Limited Search (DLS)

Depth Limited Search is a modified version of DFS that imposes a limit on the depth of the search. This means that the algorithm will only explore nodes up to a certain depth, effectively preventing it from going down excessively deep paths that are unlikely to lead to the goal. By setting a maximum depth limit, DLS aims to improve efficiency and ensure more manageable search times.

How Depth Limited Search Works

1. **Initialization:** Begin at the root node with a specified depth limit.
2. **Exploration:** Traverse the tree or graph, exploring each node's children.
3. **Depth Check:** If the current depth exceeds the set limit, stop exploring that path and backtrack.

4. **Goal Check:** If the goal node is found within the depth limit, the search is successful.
5. **Backtracking:** If the search reaches the depth limit or a leaf node without finding the goal, backtrack and explore other branches.

Applications of Depth Limited Search in AI

1. **Pathfinding in Robotics:** DLS is employed for nonholonomic motion planning of robots in the presence of obstacles. By imposing restriction on the depth it makes the robot stop after exploring a particular depth of an area and restricting the robot from too much wandering.
2. **Network Routing Algorithms:** A DLS can be implemented to compute paths between nodes in computer networks restricting the number of hops to prevent loops.
3. **Puzzle Solving in AI Systems:** DLS can be used to solve puzzles such as the 8-puzzle or Sudoku by manipulating possible moves a fixed number of times that reduces how many steps are taken in the search.
4. **Game Playing:** In AI for games, instead, DLS can be used to plan forward a few moves up to a certain level of depth to help decide how much effort to put into a given decision.

DFS can be used for various tasks, including searching for a path between two nodes, traversing a tree or graph to visit all nodes, or solving certain types of puzzles. It's often used as a building block in more complex algorithms and AI applications, such as solving mazes, analyzing game states, or exploring decision trees in search algorithms like minimax for games like chess or tic-tac-toe.

Finding Path in Robotics using Depth Limited Search Algorithm

Problem Setup

- **Grid Environment:** A 5x5 grid where 0 represents a free cell and 1 represents an obstacle.
- **Initial Position:** The starting position of the robot.
- **Goal Position:** The target position the robot needs to reach.
- **Movements:** The robot can move up, down, left, or right.

We will be using Depth Limited Search Algorithm to resolve the problem using following steps:

Step 1: Defining the grid environment

Here, a 2D list grid represents the environment where the robot will navigate. Each cell in the grid can be either a free space (0) or an obstacle (1).

```
# Define the grid environment with obstacles (1 indicates obstacle, 0
indicates free cell)
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0]
]
```

Step 2: Defining Initial and Goal Positions

The initial position of the robot is defined as (0, 0), and the goal position is set to (4, 4).

```
# Define initial and goal positions
```

```
initial_position = (0, 0)
```

```
goal_position = (4, 4)
```

Step 3: Defining Problem Representation

Now, a **Problem** class represents the problem of finding a path from the initial position to the goal position on the grid. It includes methods to check if a given state is the goal state (**is_goal**) and to expand a given state to find possible next states (**expand**).

```
# Define problem representation
```

```
class Problem:
```

```
    def __init__(self, grid, initial, goal):
        self.grid = grid
        self.initial_state = initial
```

```

self.goal_state = goal

def is_goal(self, state):
    return state == self.goal_state

def expand(self, state):
    # Define possible movements: up, down, left, right
    movements = [(0, -1), (0, 1), (-1, 0), (1, 0)]
    valid_moves = []
    for move in movements:
        new_x = state[0] + move[0]
        new_y = state[1] + move[1]
        # Check if the new position is within the grid and not an obstacle
        if 0 <= new_x < len(self.grid) and 0 <= new_y < len(self.grid[0]) and
self.grid[new_x][new_y] == 0:
            valid_moves.append((new_x, new_y))
    return valid_moves

```

Step 4: Defining Depth Limited Search (DLS) Algorithm

These functions implement the Depth Limited Search (DLS) algorithm. **depth_limited_search** is the main function that initiates the search with the given depth limit. **recursive_dls** is a helper function that performs the recursive depth-limited search. If the goal state is found, it returns the path to that state. If the depth limit is reached, it returns "cutoff". If no solution is found within the depth limit, it returns "failure".

DLS algorithm

```

def depth_limited_search(problem, depth_limit):
    return recursive_dls(problem.initial_state, problem, depth_limit)

def recursive_dls(node, problem, depth_limit):
    if problem.is_goal(node):
        return [node]
    elif depth_limit == 0:
        return "cutoff"
    else:
        for child in problem.expand(node):
            result = recursive_dls(child, problem, depth_limit - 1)
            if result == "cutoff":
                continue
            elif result != "failure":
                return [node] + result
        return "failure"

```

Step 6: Creating Problem Instance

An instance of the Problem class is created with the defined grid, initial position, and goal position.

Create problem instance

```
problem = Problem(grid, initial_position, goal_position)
```

Step 7: Finding Path using DLS

The DLS algorithm is applied to find a path from the initial to the goal position within a depth limit of 10. We also check the result of the search and prints the path if one is found. If no path is found within the depth limit, it prints a corresponding message.

Find path using DLS with depth limit 10

```
depth_limit = 10
```

```
path = depth_limited_search(problem, depth_limit)
```

```
# Output path
```

```
if path == "failure":
    print("No path found within the depth limit.")
elif path == "cutoff":
    print("Search terminated due to depth limit.")
else:
    print("Path found:", path)
```

Step 8: Visualizing the Path

The function **visualize_path** visualizes the grid with the path found by the DLS algorithm highlighted in blue. It also marks the initial position in green and the goal position in red.

```
import matplotlib.pyplot as plt
# Visualization code
def visualize_path(grid, path):
    fig, ax = plt.subplots()
    # Create a grid with the same dimensions
    nrows, ncols = len(grid), len(grid[0])
    ax.set_xticks([x - 0.5 for x in range(1, ncols)], minor=True)
    ax.set_yticks([y - 0.5 for y in range(1, nrows)], minor=True)
    ax.grid(which="minor", color="black", linestyle='-', linewidth=2)
    ax.imshow(grid, cmap='Greys', interpolation='none')

    # Highlight the path
    if path:
        for (x, y) in path:
            ax.add_patch(plt.Circle((y, x), radius=0.3, color='blue'))

    # Highlight initial and goal positions
    ax.add_patch(plt.Circle((initial_position[1], initial_position[0]), radius=0.3, color='green'))
    ax.add_patch(plt.Circle((goal_position[1], goal_position[0]), radius=0.3, color='red'))

    plt.gca().invert_yaxis()
    plt.title("Original Grid with DLS Path")
    plt.show()
```

Complete Code Finding Path in Robotics

```
import matplotlib.pyplot as plt

# Define the grid environment with obstacles (1 indicates obstacle, 0 indicates free cell)
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0]
]

# Define initial and goal positions
initial_position = (0, 0)
goal_position = (4, 4)

# Define problem representation
class Problem:
    def __init__(self, grid, initial, goal):
        self.grid = grid
```

```
self.initial_state = initial
self.goal_state = goal

def is_goal(self, state):
    return state == self.goal_state

def expand(self, state):
    # Define possible movements: up, down, left, right
    movements = [(0, -1), (0, 1), (-1, 0), (1, 0)]
    valid_moves = []
    for move in movements:
        new_x = state[0] + move[0]
        new_y = state[1] + move[1]
        # Check if the new position is within the grid and not an obstacle
        if 0 <= new_x < len(self.grid) and 0 <= new_y < len(self.grid[0]) and
self.grid[new_x][new_y] == 0:
            valid_moves.append((new_x, new_y))
    return valid_moves

# DLS algorithm
def depth_limited_search(problem, depth_limit):
    return recursive_dls(problem.initial_state, problem, depth_limit)

def recursive_dls(node, problem, depth_limit):
    if problem.is_goal(node):
        return [node]
    elif depth_limit == 0:
        return "cutoff"
    else:
        for child in problem.expand(node):
            result = recursive_dls(child, problem, depth_limit - 1)
            if result == "cutoff":
                continue
            elif result != "failure":
                return [node] + result
        return "failure"

# Create problem instance
problem = Problem(grid, initial_position, goal_position)

# Find path using DLS with depth limit 10
depth_limit = 10
path = depth_limited_search(problem, depth_limit)

# Output path
if path == "failure":
    print("No path found within the depth limit.")
elif path == "cutoff":
    print("Search terminated due to depth limit.")
else:
    print("Path found:", path)
```

```

# Visualization code
def visualize_path(grid, path):
    fig, ax = plt.subplots()
    # Create a grid with the same dimensions
    nrows, ncols = len(grid), len(grid[0])
    ax.set_xticks([x - 0.5 for x in range(1, ncols)], minor=True)
    ax.set_yticks([y - 0.5 for y in range(1, nrows)], minor=True)
    ax.grid(which="minor", color="black", linestyle='-', linewidth=2)
    ax.imshow(grid, cmap='Greys', interpolation='none')

    # Highlight the path
    if path:
        for (x, y) in path:
            ax.add_patch(plt.Circle((y, x), radius=0.3, color='blue'))

    # Highlight initial and goal positions
    ax.add_patch(plt.Circle((initial_position[1], initial_position[0]), radius=0.3, color='green'))
    ax.add_patch(plt.Circle((goal_position[1], goal_position[0]), radius=0.3, color='red'))
    plt.gca().invert_yaxis()
    plt.title("Grid with DLS Path")
    plt.show()

visualize_path(grid, path)

# Visualization of the original grid if no path found
def visualize_grid(grid):
    fig, ax = plt.subplots()
    # Create a grid with the same dimensions
    nrows, ncols = len(grid), len(grid[0])
    ax.set_xticks([x - 0.5 for x in range(1, ncols)], minor=True)
    ax.set_yticks([y - 0.5 for y in range(1, nrows)], minor=True)
    ax.grid(which="minor", color="black", linestyle='-', linewidth=2)
    ax.imshow(grid, cmap='Greys', interpolation='none')

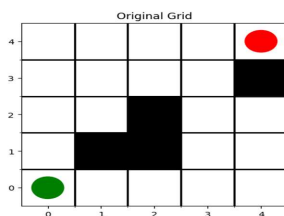
    # Highlight initial and goal positions
    ax.add_patch(plt.Circle((initial_position[1], initial_position[0]), radius=0.3, color='green'))
    ax.add_patch(plt.Circle((goal_position[1], goal_position[0]), radius=0.3, color='red'))
    plt.gca().invert_yaxis()
    plt.title("Original Grid")
    plt.show()

visualize_grid(grid)

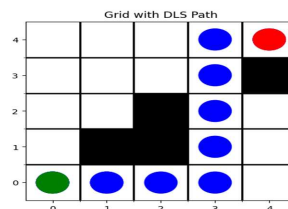
```

Output:

Path found: [(0, 0), (0, 1), (0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (3, 3), (4, 3), (4, 4)]



Original Grid Discussed in the Problem Statement



Resulting Grid After applying Depth Limit Search Algorithm